

SOFTWARE REQUIREMENTS SPECIFICATION

NSS Management Application for a B.Tech College

Version 1.0

Document Item	Value
Project Name	NSS College Management Application
Institution Type	B.Tech / Engineering College
Document Type	Software Requirements Specification (SRS)
Primary Users	NSS Volunteers, Student Leaders, Programme Officers, Faculty Coordinators, Administrators
System Style	Responsive Web Application with REST API
Status	Baseline specification for design and development

Purpose: Define the functional, non-functional, data, security, interface, operational and acceptance requirements for a college-level National Service Scheme (NSS) management application.

1. Introduction

1.1 Purpose

This Software Requirements Specification defines the requirements for an NSS management application intended for a B.Tech college. The system will centralize volunteer records, academic information relevant to NSS administration, NSS units, events, registrations, attendance, service hours, activity reports, documents, announcements, achievements and administrative reporting.

1.2 Scope

The application will replace or reduce fragmented workflows based on paper registers, spreadsheets and informal communication channels. It will provide a controlled digital workflow from volunteer onboarding through event participation, verified attendance, service-hour calculation and reporting.

1.3 Intended Audience

- B.Tech students and NSS volunteers.
- NSS Programme Officers and faculty coordinators.
- Student NSS leaders.
- College administrators and authorized system administrators.
- Project developers, testers and maintainers.
- Project reviewers, faculty guides and institutional stakeholders.

1.4 Definitions

Term	Definition
NSS	National Service Scheme.
Volunteer	A student enrolled and approved as an NSS volunteer.
NSS Unit	An organizational NSS group managed under the college's NSS structure.
Programme Officer	Authorized faculty member responsible for NSS programme administration.
Service Hours	Verified hours of NSS participation recorded according to institutional rules.
Activity Report	Official post-event record describing an NSS activity, participation and outcomes.
RBAC	Role-Based Access Control.
Audit Log	Tamper-resistant record of important administrative or security-relevant actions.

2. Overall Description

2.1 Product Perspective

The NSS application will operate as a centralized college application. The frontend will communicate with a backend API over HTTPS. The backend will enforce business rules, authorization and validation. PostgreSQL will store transactional records, while object storage will hold photographs and documents.

2.2 High-Level Organization

The system shall model the college hierarchy as College → Department → Academic Batch/Year → NSS Unit → Volunteer. A volunteer may participate in events administered by an eligible NSS unit according to institutional rules.

2.3 Major Product Functions

- Identity, authentication and authorization.
- College, department and NSS unit administration.
- Volunteer enrollment, verification and profiles.
- Event creation, publishing, registration and lifecycle management.
- Attendance session management and verified check-in.
- Service-hour calculation and auditable history.
- Activity reports and document/photo management.
- Announcements and notifications.
- Achievements and certificate records.
- Dashboards, search, filtering and exports.
- Administrative audit logging and system configuration.

2.4 Assumptions

- The college will define official NSS rules, service-hour policies and certificate policies.
- Only authorized institutional personnel will administer official NSS records.
- Users have access to a modern browser and an internet or college-network connection.
- The college may provide an institutional identity provider in the future.
- Personal-data collection will be limited to information required for NSS operations.

2.5 Constraints

- The system must preserve historical records needed for institutional reporting.
- Authorization must be enforced server-side.
- Attendance records must be protected against unauthorized modification.
- The system should be maintainable by a small technical team.
- The first release should avoid unnecessary microservice complexity.

3. User Roles and Permissions

Role	Primary Capabilities
Volunteer	Manage permitted profile fields, browse events, register, check attendance status, view service history, announcements and approved achievements.
Student Leader	Assist with assigned event operations, participant coordination and attendance workflows within delegated permissions.
Programme Officer	Manage assigned NSS units, volunteers, events, registrations, attendance, reports, announcements and service records.
Faculty Coordinator	Coordinate across assigned units/departments and review operational reports as authorized.
System Administrator	Manage users, roles, college structure, configuration, audit access and system-level administration.

Principle: permissions must be based on least privilege and scope. A coordinator assigned to one NSS unit must not automatically gain access to unrelated units.

4. Functional Requirements

ID	Requirement	Description
FR-001	User Authentication	The system shall authenticate users using an approved institutional authentication method or secure application credentials.
FR-002	Session Security	The system shall provide secure session/token handling, logout and expiration controls.
FR-003	Role Authorization	The system shall enforce role-based and resource-level authorization on protected operations.
FR-004	College Structure	Administrators shall manage departments, academic batches/years and NSS units.
FR-005	Volunteer Enrollment	Authorized users shall create, import or approve volunteer records according to the college workflow.
FR-006	Volunteer Profile	The system shall maintain volunteer information including name, student identifier, department, programme/year, contact information and NSS unit membership where applicable.
FR-007	Profile History	The system shall preserve relevant historical changes to unit membership and status.
FR-008	Event Creation	Authorized coordinators shall create NSS events with title, description, date/time, venue, organizer, eligibility and capacity where applicable.
FR-009	Event Lifecycle	Events shall support controlled states such as Draft, Published, Open, Closed, Completed, Cancelled and Archived.
FR-010	Event Registration	Eligible volunteers shall be able to register for published events when registration is enabled.
FR-011	Registration Rules	The system shall prevent duplicate registrations and enforce capacity/eligibility constraints.
FR-012	Attendance Session	Authorized coordinators shall open and close controlled attendance sessions for events.
FR-013	Attendance Check-In	The system shall support verified attendance through an approved mechanism such as authenticated QR-based check-in or coordinator marking.
FR-014	Duplicate Prevention	The system shall prevent multiple valid attendance records for the same volunteer and attendance session.
FR-015	Attendance Correction	Authorized attendance corrections

		shall require a reason and produce an audit record.
FR-016	Service Hours	The system shall calculate or record service hours only from verified/approved participation according to configured rules.
FR-017	Service Ledger	Service-hour changes shall be traceable to the underlying event/attendance and correction history.
FR-018	Activity Report	Authorized users shall create post-event reports containing objectives, activities, outcomes, participant information and remarks.
FR-019	Document Management	Authorized users shall upload and associate photographs, PDFs and other approved documents with relevant entities.
FR-020	Announcements	Authorized coordinators shall publish announcements to defined audiences such as units or all volunteers.
FR-021	Notifications	The system shall generate relevant notifications for events, registration status, attendance status and announcements.
FR-022	Achievements	Authorized users shall record approved awards, leadership roles, recognitions and other achievements.
FR-023	Certificates	The system shall store certificate records and may generate approved certificate documents according to institutional templates.
FR-024	Search and Filtering	Users shall be able to search and filter records according to their permissions.
FR-025	Dashboards	The system shall provide role-specific dashboards with operational statistics and pending actions.
FR-026	Reports	Authorized users shall generate attendance, participation, service-hour and activity reports.
FR-027	Export	Authorized users shall export permitted report data in approved formats such as CSV or PDF.
FR-028	Audit Logging	The system shall record important administrative, authorization and attendance-related actions.
FR-029	Data Retention	The system shall support institutional retention and archival rules without silently destroying required historical records.
FR-030	Administration	System administrators shall manage configuration, roles, user status, unit assignments and security-related settings.

5. Detailed Module Requirements

5.1 Identity & Access

- Login, logout and secure session handling.
- Role assignment and revocation.
- Account activation/deactivation.
- Password reset if local credentials are used.
- Authorization at both UI and API layers.
- Audit of privileged identity changes.

5.2 College & Academic Structure

- Department records such as CSE, ECE, EEE, Mechanical, Civil and other college-approved branches.
- Academic year/batch and year/semester metadata.
- Mapping of volunteers to their academic department and NSS unit.
- Historical membership where required for reporting.

5.3 Volunteer Management

- Volunteer registration and approval.
- Profile viewing and controlled editing.
- Volunteer status such as Pending, Active, Inactive or Archived.
- Unit assignment and leadership designation.
- Participation and service-history view.

5.4 Event Management

- Create, edit, publish and cancel events.
- Event categories such as awareness programme, cleanup, blood donation drive, rally, camp, meeting or other institution-defined categories.
- Registration window, participant limit and eligibility rules.
- Event participant list and operational status.

5.5 Attendance

- Open attendance session for an event.
- Support authenticated QR check-in or coordinator verification.
- Prevent duplicate check-ins.
- Record timestamp and attendance source.
- Allow controlled corrections with mandatory reason and audit.

5.6 Service Hours

- Calculate hours from verified participation.
- Support institution-defined caps, exclusions or manual approval if required.
- Show volunteer totals and event-level contribution.
- Keep an auditable ledger instead of relying only on a mutable total field.

5.7 Reports & Documents

- Create post-event reports.

- Attach approved documents and photographs.
- Restrict private files through authorization and signed access.
- Provide report review/approval states where required.

5.8 Communication

- Publish announcements to defined audiences.
- Send in-app notifications.
- Support optional email notifications if configured.
- Record notification delivery status where supported.

5.9 Analytics & Exports

- Unit-wise, department-wise and event-wise participation.
- Attendance and service-hour summaries.
- Volunteer activity history.
- Date-range filtering.
- CSV/PDF export subject to permissions.

5.10 Administration & Audit

- Manage roles and permissions.
- Manage college structure and NSS units.
- Review audit events.
- Configure approved event categories and system policies.
- Manage archival and retention settings.

6. Core Business Workflows

6.1 Volunteer Onboarding

1. Student submits registration or is imported by an authorized administrator.
2. System validates required academic and identity information.
3. Coordinator reviews eligibility and approves or rejects the record.
4. Approved volunteer is assigned to an NSS unit.
5. Volunteer receives confirmation and can access the volunteer dashboard.

6.2 Event Participation

6. Coordinator creates event in Draft state.
7. Coordinator publishes the event and opens registration if required.
8. Eligible volunteers register.
9. Coordinator closes registration or reaches capacity.
10. Event occurs and an attendance session is opened.
11. Attendance is verified.
12. Coordinator closes attendance.
13. Verified attendance contributes to the service-hour ledger.
14. Coordinator creates and submits the activity report.

6.3 Attendance Correction

15. Authorized user selects an attendance record.
16. System requires the correction reason.
17. System verifies the actor has permission.

18. Original state is preserved in audit history.
19. Corrected state is stored transactionally.
20. Any affected service-hour totals are recalculated or adjusted through the ledger.

7. Non-Functional Requirements

ID	Category	Requirement
NFR-001	Performance	Normal interactive API requests should target p95 latency below 500 ms under the designed baseline workload, excluding large file generation/download operations.
NFR-002	Scalability	The API shall support horizontal scaling without relying on process-local session state.
NFR-003	Availability	Production deployment should use managed services and automated recovery mechanisms appropriate to college operational requirements.
NFR-004	Security	The system shall follow secure authentication, authorization, input validation, transport encryption and secret-management practices.
NFR-005	Privacy	The system shall minimize collection and exposure of personal information and follow college-approved privacy/retention policies.
NFR-006	Auditability	Sensitive administrative and attendance operations shall be auditable.
NFR-007	Maintainability	Backend modules shall have explicit domain boundaries and automated tests for critical business rules.
NFR-008	Usability	The interface shall be responsive and usable on common desktop and mobile browser sizes.
NFR-009	Accessibility	The application should follow WCAG-aligned accessibility practices, including keyboard navigation, labels and non-color-only status communication.
NFR-010	Reliability	Critical transactional operations shall use database transactions and appropriate constraints.
NFR-011	Backup	Production data shall have automated encrypted backups and a documented restoration procedure.
NFR-012	Observability	The application shall provide structured logs, health checks and metrics sufficient to diagnose production failures.

8. System Architecture Requirements

8.1 Recommended Architecture

The first production version should use a modular monolith. This is preferable to microservices for a single B.Tech college because the workload and organizational scope do not justify distributed-service operational complexity.

Recommended stack: React + TypeScript frontend; NestJS + TypeScript backend; PostgreSQL database; Prisma ORM; S3-compatible object storage; Redis/worker queue for asynchronous jobs when required; containerized deployment with managed infrastructure.

8.2 Logical Layers

- Presentation layer: responsive web UI and role-specific navigation.
- API layer: HTTP routing, authentication, validation, rate limiting and request/response contracts.
- Application layer: use cases and orchestration.
- Domain layer: event, attendance, volunteer, service-hour and authorization rules.
- Persistence layer: PostgreSQL repositories and transactional data access.
- Infrastructure layer: object storage, notifications, background jobs, logging and external identity services.

8.3 Database Requirements

- Use PostgreSQL as the authoritative transactional store.
- Use foreign keys and unique constraints to protect referential integrity.
- Use indexes on event dates/status, volunteer identifiers, unit IDs, registration and attendance keys.
- Use migrations under version control.
- Use transactions for attendance verification and service-ledger updates.

9. Conceptual Data Model

Entity	Key Information
User	Identity, authentication state, account status.
Role	Role name and permission mapping.
Department	Engineering department/branch information.
AcademicBatch	Academic year/batch metadata.
NSSUnit	NSS unit name, status and coordinator assignment.
VolunteerProfile	Student identity, academic information and NSS profile.
UnitMembership	Volunteer-to-unit relationship and history.
Event	Title, type, schedule, venue, eligibility, capacity and status.
EventRegistration	Volunteer registration and status.
AttendanceSession	Event attendance window and controls.
AttendanceRecord	Verified/pending attendance, timestamp and source.
ServiceHourLedger	Auditable service-hour entries and adjustments.
ActivityReport	Objectives, description, outcomes, participants and approval state.
Document	File metadata, storage key, owner and associated entity.
Announcement	Message, audience, publication and expiry.
Notification	Recipient, type, status and delivery metadata.
Achievement	Award/recognition/leadership record.
Certificate	Certificate type, recipient, status and file reference.
AuditLog	Actor, action, resource, timestamp and safe metadata.

Conceptual relationship: Department → AcademicBatch → VolunteerProfile; NSSUnit → UnitMembership → VolunteerProfile; Event → EventRegistration → VolunteerProfile; Event → AttendanceSession → AttendanceRecord → ServiceHourLedger; Event → ActivityReport → Document.

10. External and Internal Interfaces

10.1 REST API

Resource	Representative Operations
Authentication	POST /api/v1/auth/login; POST /api/v1/auth/logout; POST /api/v1/auth/refresh
Volunteers	GET /api/v1/volunteers; GET/PATCH /api/v1/volunteers/{id}
Units	GET/POST /api/v1/units; PATCH /api/v1/units/{id}
Events	GET/POST /api/v1/events; PATCH /api/v1/events/{id}
Registrations	POST /api/v1/events/{id}/registrations; DELETE /api/v1/events/{id}/registrations/{userId}
Attendance	POST /api/v1/events/{id}/attendance/session; POST /api/v1/attendance/check-in; PATCH /api/v1/attendance/{id}
Reports	POST /api/v1/events/{id}/report; GET /api/v1/reports
Documents	POST /api/v1/documents/upload-url; GET /api/v1/documents/{id}
Announcements	POST/GET /api/v1/announcements
Analytics	GET /api/v1/dashboard; GET /api/v1/reports/attendance

10.2 File Interface

Files should be uploaded directly to private object storage using short-lived signed upload URLs where feasible. The API stores only validated metadata and the opaque storage key. Downloads should be authorized and use short-lived signed URLs.

11. Security Requirements

- All production traffic shall use HTTPS/TLS.
- Server-side authorization shall be applied to every protected resource.
- The system shall validate all request parameters, bodies, headers and uploaded-file metadata.
- The system shall use parameterized database operations or a safe ORM interface.
- Authentication endpoints shall have rate limiting and abuse protection.
- Secrets shall not be committed to source control.
- Sensitive data shall not be written to ordinary logs.
- Private documents shall not be publicly readable by default.
- Uploaded files shall have explicit size/type restrictions and should be malware-scanned where infrastructure permits.
- Attendance check-in tokens, if QR is used, shall be short-lived and server-generated; static reusable attendance QR codes shall not be treated as authoritative.
- Administrative changes to roles, attendance and volunteer status shall be audited.
- Dependencies shall be regularly updated and checked for known vulnerabilities.

12. Backup, Recovery and Disaster Management

- Automated encrypted database backups shall be configured for production.
- Point-in-time recovery should be enabled when supported by the selected managed database.
- Important object storage should use versioning/protection appropriate to institutional requirements.
- Restoration procedures shall be documented.
- Restoration drills should be performed periodically.
- The college should define Recovery Time Objective (RTO) and Recovery Point Objective (RPO).

13. Testing Requirements

Test Level	Required Coverage
Unit	Service-hour calculations, eligibility rules, event states, permission policies, validation.
Integration	Database constraints, transactions, registration, attendance and audit creation.
API	Authentication, authorization, validation, pagination, errors, idempotency.
End-to-End	Volunteer onboarding → event registration → attendance → verified service hours → report.
Security	Broken access control, IDOR, injection, rate-limit bypass, unsafe upload and session issues.
Performance	Representative event registration, attendance and dashboard/report workloads.
Recovery	Database backup restoration and object/document recovery procedures.

13.1 Boundary Cases

- Duplicate registration.
- Duplicate or concurrent attendance check-in.
- Expired attendance session.
- Cancelled event with existing registrations.
- Volunteer removed from a unit after registration.
- Unauthorized cross-unit access.
- Invalid event duration or invalid dates.
- Oversized or unsupported document.
- Attempt to modify verified attendance without permission.
- Service-hour adjustment that would produce an invalid total.

14. Acceptance Criteria

- A verified volunteer can log in and view only permitted data.
- An authorized coordinator can create and publish an event.
- Eligible volunteers can register without duplicate records.
- Attendance can be opened, verified and closed with duplicate protection.
- Unauthorized users cannot alter attendance or access restricted unit data.
- Verified participation updates the service-hour ledger according to configured rules.
- Attendance corrections require authorization and leave an audit trail.
- An activity report can be associated with an event and include approved documents.
- Authorized users can generate participation and service-hour reports.

- The application remains usable on desktop and mobile browser widths.
- Production secrets are not exposed in source code or logs.
- Automated tests cover critical domain and authorization rules.

15. Future Enhancements

- Native Android/iOS application or progressive web app enhancements.
- College SSO integration.
- Multi-campus or multi-college tenancy.
- Advanced analytics and trend dashboards.
- Automated certificate generation with institutional templates.
- Offline-friendly attendance workflows where connectivity is unreliable.
- Calendar integration.
- Institution-approved SMS/email gateway integration.
- Public-facing NSS activity portal.

16. Requirements Traceability Matrix

Business Goal	Related Requirements	Validation
Centralize volunteer management	FR-004 to FR-007	Volunteer/unit integration tests + UAT
Reliable event management	FR-008 to FR-011	API/E2E tests
Trusted attendance	FR-012 to FR-017	Security + concurrency + integration tests
Institutional reporting	FR-018, FR-026, FR-027	Report/UAT tests
Secure communication	FR-020, FR-021	Authorization + delivery tests
Governance and accountability	FR-028 to FR-030	Audit/security/UAT

17. Architectural Trade-Offs

Memory: A modular monolith with PostgreSQL and optional Redis/worker components has a moderate operational memory footprint; object storage prevents large media files from consuming database resources.

Latency: Keeping domain operations within one backend minimizes network hops. Indexed PostgreSQL queries and asynchronous background jobs should keep interactive operations responsive.

Operational Overhead: Managed PostgreSQL/object storage and a small number of deployable components reduce administration compared with microservices, while backups, audit logging and security monitoring add necessary institutional overhead.

18. Approval and Change Control

This SRS should be reviewed by the NSS Programme Officer, faculty/project guide and authorized college stakeholders before implementation. Any changes to official service-hour rules, attendance policy, certificate validity, personal-data retention or institutional reporting requirements should be reflected in a versioned revision of this document.

Role	Name	Signature	Date
NSS Programme Officer			

Faculty / Project Guide			
College Administrator			
Project Team Representative			

End of SRS — Version 1.0